SKernel: An Elastic and Efficient Secure Container System at Scale with a Split-Kernel Architecture

Xiaohu Chai[1,2], Keyang Hu[1], Jianfeng Tan[2], Tiwei Bie[2], Guotao Tan[2], Tianyu Zhou[2], Anqi Shen[2], Dawei Shen[2], Xinyao Yang[2], Xin Chen[2], Xu Wang[2], Feng Yu[2], Zhengyu He[2], Dong Du[3], Yubin Xia[3], Kang Chen[1], Yu Chen[4,1]

[1]Tsinghua University [2]Ant Group [3]Shanghai Jiao Tong University [4]Quan Cheng Laboratory

Abstract

Secure containers leverage hardware virtualization to isolate container sandboxes, enabling dedicated guest kernels to mitigate shared kernel attacks prevalent in traditional systems. However, existing approaches struggle with a fundamental trade-off: VM-based solutions (e.g., Kata) prioritize performance but lack elasticity and on-demand usage for volatile and bursty workloads, while lightweight methods (e.g., gVisor) rely on the host kernel for dynamic resource management at the cost of significant performance degradation due to guest-host dependencies.

We present SKernel, a production-proven secure container system that resolves this trade-off through a novel *split-kernel* architecture. Observing I/O stacks dominate performance for cloud-native apps, SKernel decouples the guest kernel into two components: *SKernel-R* (resource management via host collaboration) and *SKernel-D* (high-performance I/O operations without host involvement). Key innovations of SKernel include (1) a fast syscall forwarding approach that minimizes context switch overheads; (2) a device passthrough-based network stack that offers improved compatibility and resource flexibility, combined with an advanced thread model and I/O notification mechanism to handle volatile and bursty workloads; (3) a file descriptor-based container filesystem that enhances security, eliminates unnecessary indirection, and reduces costs. Macrobenchmarks evaluations show SKernel achieves 4.49× throughput gains over gVisor while matching its elasticity. SKernel reduces CPU usage compared to runc under production workloads, supporting over 2,000 long-running microservices applications across 40,000+ nodes (with 100,000+ secure containers) at Ant Group. SKernel represents a milestone as a production-ready secure container that delivers both efficiency and elasticity.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → **Operating systems**.

This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, Edinburgh, Scotland Uk

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04
https://doi.org/10.1145/3767295.3769332

Keywords: Operating systems, Cloud computing, Virtualization, Secure Containers

ACM Reference Format:
Xiaohu Chai, Keyang Hu, Jianfeng Tan, Tiwei Bie, Guotao Tan, Tianyu Zhou, Anqi Shen, Xinyao Yang, Xin Chen, Xu Wang, Feng Yu, Zhengyu He, Dong Du, Yubin Xia, Kang Chen and Yu Chen. 2026. SKernel: An Elastic and Efficient Secure Container System at Scale with a Split-Kernel Architecture. In *European Conference on Computer Systems (EUROSYS '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3767295.3769332

1 Introduction

The rapid adoption of cloud-native technologies such as Kubernetes [61], containers [26], and service mesh [11] has transformed how applications are developed and deployed in public clouds [10, 69]. Applications are increasingly split into lightweight, independent components like microservices and serverless functions. Containers, which package these components into portable and isolated units, have become the backbone of cloud-native ecosystems. As shown in Figure 1, a typical application runs in a dedicated container and uses sidecar proxies to communicate with other services.

Traditional containers like Docker [26] and LXC [76] face security risks due to reliance on a *shared host kernel* [98]. To address this, two solutions have emerged: secure containers (e.g., Firecracker [8], gVisor [42]) and confidential containers using trusted execution environments (TEEs) [18, 90]. Secure containers provide stronger isolation by assigning each container its *own guest kernel*, eliminating the risks of kernel sharing while preserving the simplicity of container workflows. Unlike TEE-based solutions, secure containers are widely adopted in production as they use mature hardware virtualization for better performance and practicality.

As a major cloud provider, Ant Group also rely on secure containers to host multi-tenant applications, spanning long-running microservices and ephemeral serverless workloads [23, 84]. Our experience reveals four critical requirements for secure containers in production: ***(1) Isolation:*** Secure containers must enforce both *pod-level* (inter-pod) and *container-level* (intra-pod) isolation, as pods often contain containers from multiple providers (e.g., app and sidecar containers). ***(2) Resource Elasticity:*** Microservices require

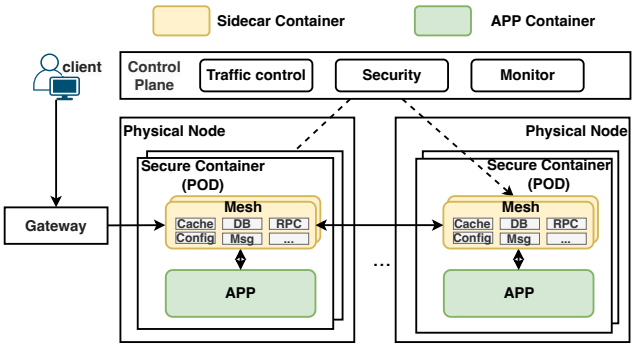

Figure 1. Microservice architecture in Ant Group (solid lines for data path, dashed lines for control path). A pod [62] is deployed within one secure container. Typically, a pod consists of one app container and multiple sidecar containers [77]. Applications and sidecar services, developed and maintained by distinct providers, should be isolated for security.

dynamic scaling. Secure containers must adjust vCPUs, memory, and I/O resources in real time to match workload demands. ***(3) Performance:*** Meeting strict SLAs [39] demands near-native performance for compute, network, and storage operations. ***(4) Compatibility:*** Support for unmodified applications (e.g., Java, Go, Python, JS) and standard container images is essential to avoid vendor lock-in.

Despite strong security and compatibility, existing secure containers face a fundamental trade-off between elasticity and performance. Two approaches dominate: ***(1) MicroVM-based containers*** (e.g., Kata [29]) isolate workloads via hardware virtualization with dedicated guest kernels. While delivering near-VM performance by minimizing host involvement, their rigid resource management (e.g., static vCPU/memory allocation) struggles with dynamic cloud workloads. Workarounds like memory ballooning introduce latency incompatible with sub-second scaling demands in microservices. ***(2) Process virtualization*** (e.g., gVisor [42]) uses a lightweight guest kernel (Sentry) that depends on the host for resource operations. This enables fine-grained elasticity but incurs frequent guest-host context switches, significantly degrading performance in Ant Group 's production environments (details in §2.3). Thus, while process virtualization suits elastic platforms, its performance overhead limits adoption for performance-sensitive workloads.

To meet elasticity demands, we adopted gVisor in Ant Group as the secure container system, and identify three critical performance challenges after years of production use: ***Challenge-1: Syscall Forwarding Overhead.*** gVisor's dual syscall costs stem from: ① *L1 (App→Guest Kernel)*: Syscall overhead from user applications to guest kernel, caused by Go's memory-safe implementation (security vs. performance trade-off). ② *L2 (Guest→Host Kernel)*: Hypercall overhead from guest to host kernel due to frequent context switches and lengthy call paths. ***Challenge-2: Network I/O Bottlenecks.*** Cloud-native apps often require high network throughput, but traditional kernel-bypass solutions (e.g., DPDK [3]) conflict with elasticity by needing dedicated cores or pinned memory [24]. Others sacrifice compatibility (requiring app rewrites [110]) or isolation (libOS approaches [30]). ***Challenge-3: Filesystem I/O Penalty.*** While gVisor's Gofer proxy mitigates host filesystem risks, file I/O operations suffer from excessive VM exits and inter-process communication (IPC), degrading performance.

This paper presents SKernel, a production-ready secure container system that eliminates the elasticity-performance trade-off via a novel *split-kernel* architecture. Analysis of Ant Group 's traces shows that I/O operations dominate runtime costs for cloud-native apps like microservices. Thus, SKernel splits the guest kernel into two specialized components:

- *SKernel-R (Resource Kernel)*: For resources requiring fine-grained management (e.g., CPU and memory), SKernel-R collaborates with the host kernel to achieve globally optimal resource elasticity.
- *SKernel-D (Data Kernel)*: For performance-critical resources such as I/O, SKernel-D manages them directly within the guest kernel to achieve higher throughput and lower latency.

This split allows SKernel to deliver both VM-like performance and process-like elasticity while maintaining backward compatibility and hardware-enforced isolation.

Specifically, we design three techniques, following the split-kernel architecture, to overcome the three performance challenges in SKernel. (1) For ***Challenge-1***, we design a syscall fast path for SKernel-D to improve the performance-critical I/O-related L1 syscalls performance and to shorten the call path for L2 syscalls by using the hypervisor as a delegator. (2) For ***Challenge-2***, we design a high-performance network protocol stack tailored for cloud-native workloads in SKernel-D, which addresses the compatibility issues (multi-process, multi-threading, multi-language) and resource flexibility issues (spinning cores, pinned memory, requiring application rewriting) introduced by device passthrough, and adopts a hybrid thread model and adaptable I/O notification mechanism to match fluctuating demand. (3) For ***Challenge-3***, we enhanced both performance and security of the file system in SKernel-D by eliminating the IPC-based Gofer process and limiting interactions with the host kernel.

Our evaluation shows that SKernel exhibits a significant performance improvement compared with gVisor. SKernel achieves a performance improvement of 1.68× in the throughput for long-running microservice workloads, and 1.81× to 4.49× for macro-benchmarks such as Redis, Nginx and MySQL. In the production environment, SKernel's CPU usage is [−0.53%, −3.99%] compared to runc for real-world RPC workloads during peak load under the same level of

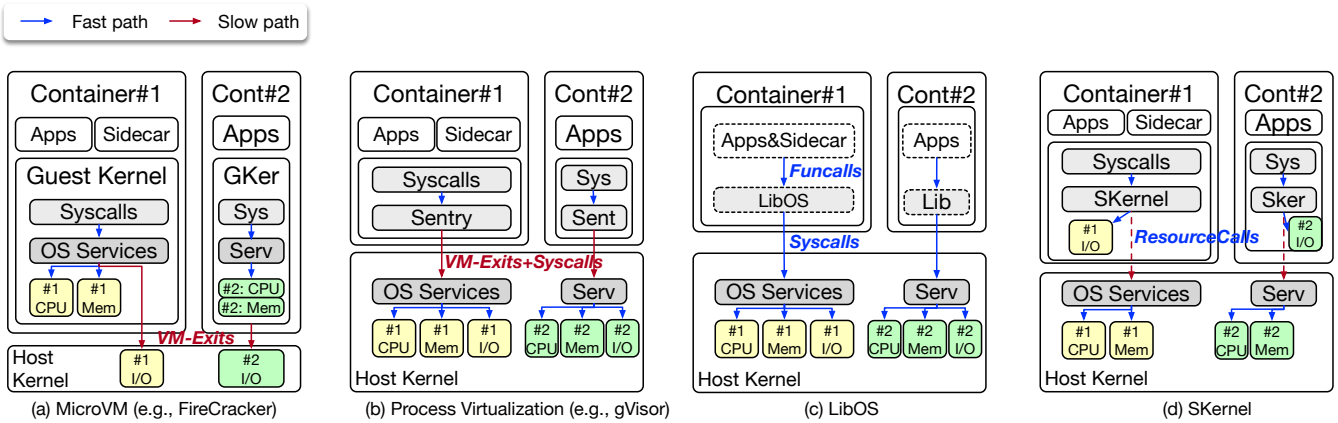

Figure 2. Comparison of different secure container architectures. (a) (micro)VM: Poor resource elasticity arises because the guest retains full control over allocated resources without further host intervention. (b) Process virtualization: Fine-grained elasticity but frequent context switches and long call paths degrade performance. (c) LibOS: Poor compatibility and isolation without split privilege levels. (d) SKernel: Efficiency and elasticity by decoupling resource management (host-coordinated) and performance-critical I/O operations (guest-local).

request. In contrast, gVisor consumes +52.47% more CPU than runc (Kata consumes +19.33% more CPU than runc).

Production at scale: SKernel has been deployed on a large scale to run multi-tenant services on the public cloud in Ant Group. More than 40, 000 physical nodes are hosting over 100, 000 secure containers daily, supporting over 2, 000 long-running microservices applications. SKernel also supports serverless scenarios and serves as the backbone of AFaaS [23]. It represents a milestone as a production-ready secure container that delivers both efficiency and elasticity.

2 Characterizing Secure Container at Scale

We begin by profiling real-world cloud-native applications and deriving design implications that motivate our adoption of process-level virtualization (gVisor) as the baseline. We then conduct an in-depth performance analysis of gVisor, identifying critical bottlenecks that directly inform SKernel's split-kernel architecture. To our knowledge, this work provides the first empirical analysis of secure containers from an industry perspective, bridging the gap between academic research and production-scale constraints.

2.1 Elasticity is Essential for Secure Container

Workloads. Ant Group supports a diverse ecosystem across finance, payments, and digital services. The applications of Ant Group include the backend applications (microservices at 48.45%, AI at 3.95%, and packages at 2.3%), web applications at 40.38%, big data at 4.43%, and terminal applications. The microservices are primarily built using SOFABoot, a Java microservices framework [92]. This reflects a broader trend in the industry [84]. According to JetBrains 2022 research [49], 86% of system designs utilize microservices, with Java being the most popular programming language (34%). Major technology companies such as Uber [28], Twitter [96], and PayPal [43] build their microservices on the JVM. They have also contributed numerous frameworks for developing JVM-based microservices, including Spotify's Apollo [95], Netflix OSS [78], and Ant Group's SOFABoot framework [92].

The deployment mode for each microservices application is shown in Figure 1. Each pod (secure container) includes applications and various infrastructure sidecars. About 30-50 distinct applications are deployed per node. Typical execution times for these microservices range from 50ms to 200ms, depending on the code and whether requests are cross-region.

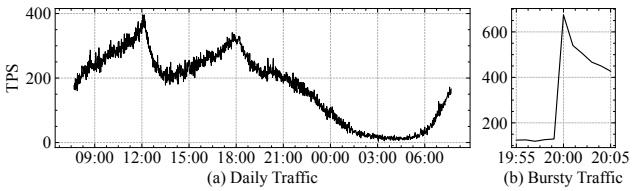

Figure 3. Traces of E-Commerce microservices. The transactions per second (TPS) handled by a single application instance under (a) normal daily conditions and (b) shopping festival peak periods.

Characteristics-1: Real-world workload behaviors. Figure 3 shows the characteristics of a typical RPC services application (a long-running Java-based e-Commerce microservice) in Ant Group. Traffic can be classified into two types based on user behavior: daily patterns and burst patterns during shopping festivals. Daily patterns require each application to handle about 400 transactions per second (TPS) during peak times, with around 10 TPS during idle periods. In contrast, burst patterns see TPS surge nearly sixfold in the first second before a promotional event. To handle these dynamics, secure containers must scale horizontally (adding

Table 1. Comparison of the benefits of previous work and SKernel.(√is beneficial, √†is partly beneficial, and blank is limited.)

Type	Related work	Compatibility	Isolation	Resource elasticity	Performance
MicroVM	RunD[71], Firecracker[8], Kata[29]	√	√		√†
Process Virtualization	gVisor[42], Dune[21]	√	√	√	
LibOS&unikernel	Junction [30], X-Containers [91], Drawbridge [86]	√†		√	√
	Mirage[73], Unikraft [64], Nabla[105], LightVM[74]		√		√
Split-Kernel (this paper)	**SKernel**	√	√	√	√

pods) and vertically (adjusting CPU/memory per pod) in real time. This dual elasticity requirement—common to both secure and traditional containers—aligns with industry practices [12, 34, 53].

> **Implication-1:** *Workload volatility and burstiness inherently require secure containers to provide* ***fine-grained elasticity****, enabling dynamic adjustment of both pod instances (horizontal) and per-pod resources (e.g., CPU/memory) to match fluctuating demand.*

We conduct an in-depth survey of various alternative secure container architectures from both academic research and industrial systems, assessing their suitability for deployment in large-scale industrial production environments. Figure 2 illustrates existing secure container designs, with a comparison in Table 1.

(1) MicroVM. (Micro)VM-based methods (e.g., Kata [29], Firecracker [8], RunD [71]) ensure full compatibility by utilizing an independent guest kernel, as shown in Figure 2(a). The guest and host are loosely coupled, minimizing host involvement. The host OS allocates physical resources into fixed quotas and employs virtualization technology to emulate a complete machine. The guest kernel takes ownership of its allocated resources, and it is difficult for the host kernel to reallocate or reclaim them once assigned. It reduces the direct dependency between guest and host, which is beneficial for performance but limits resource elasticity [14, 21, 104]. For example, vCPU hotplug is complex in QEMU/KVM [65], and virtio-balloon-based [101] memory reclamation is 10–60 times less efficient than direct reclamation [104].

(2) Process Virtualization. As shown in Figure 2(b), an alternative approach uses virtualization hardware to provide a process abstraction rather than a machine abstraction (e.g., Dune [21], gVisor[42]). The guest kernel (called Sentry in gVisor) acts as a shim layer between apps and host kernel, relying on the host kernel for global resource management. This design minimizes the resource footprint and enhances elasticity. We compare gVisor's key elasticity metrics with Kata [29] (a representative VM-based secure container) in Table 2 [1]. gVisor achieves better startup latency (424*ms* vs. 1433*ms*) and a smaller memory footprint (27*MB* vs. 51*MB*). vCPU hotplug in gVisor is straightforward, requiring only the cloning of a new vCPU thread without exposing ACPI/MPTable information or relying on hardware boot protocols like UEFI/BIOS (266*us* vs. 53*ms*). Additionally, gVisor proactively collects unused memory via hypercalls and releases it directly to the host kernel, making it more efficient than the virtio-balloon mechanism (1.33*s* vs. 5*s*). However, although the tightly-coupled host-guest kernel design in gVisor achieves great elasticity, it also incurs significant performance slowdown due to the frequent guest-host interactions (details in §2.3).

[1]The experimental setup in this section is the same as in the evaluation section (§6).

Table 2. Elasticity metrics: Boot Time (BT), vCPU Hotplug Latency (vCPU-H), Memory Footprint (Mem-F), 1GB Memory Allocation (Mem-A), and 1GB Memory Reclaim (Mem-R)

Runtime	BT	vCPU-H	Mem-F	Mem-A	Mem-R
Kata	1433ms	53ms	51MB	520ms	5s
gVisor	424ms	21.7us–266us	27MB	542ms	1.33s

(3) LibOS and Unikernel. LibOS [30] (as shown in Figure 2(c)) and Unikernel [57, 64, 73, 81] face significant adoption barriers in production environments such as Ant Group. Both approaches aim to deliver high performance together with partial *POSIX-like* compatibility [30, 81]. In both designs, application and kernel code reside within a single address space and execute at the same privilege level. LibOS systems (e.g., Junction [30]) employ kernel-bypass techniques to handle I/O directly in userspace, while Unikernels run application and kernel code together in privileged mode. Despite their performance benefits, both designs lack support for multiple address spaces, provide no isolation between applications and sidecars (discussed in §6.7), require invasive `glibc` modifications, and do not support POSIX `fork()` semantics. Combined with immature tooling ecosystems, these limitations hinder their large-scale deployment in production, particularly for security-critical and multi-tenant workloads.

> **Implication-2:** *Prevailing approaches face a fundamental* ***trade-off****: VM-based solutions (e.g., Firecracker) prioritize performance but lack elasticity and on-demand usage, while process-level virtualization (e.g., gVisor) offers near-native elasticity but incurs a performance penalty.*

2.2 gVisor Explained

By providing Linux ABI compatibility, a two-layer isolation mechanism (a minimal Golang kernel and restricted syscalls with ring protection), and a smaller and flexible resource footprint, gVisor emerges as an optimal choice for secure

containers. It has been proven in production and widely adopted by many companies (e.g. Google, OpenAI, and Ant Group) [36]. This motivates our in-depth performance analysis in the following sections.

gVisor implements a userspace kernel called Sentry, written in Go. Sentry intercepts application system calls and emulates the Linux kernel's API. It relies on syscall interfaces provided by the host kernel and uses KVM [58] to perform `VM-exit` to `host ring3` when requesting services. Essentially, Sentry operates as a user process that delegates most system management tasks to the host kernel via syscalls.

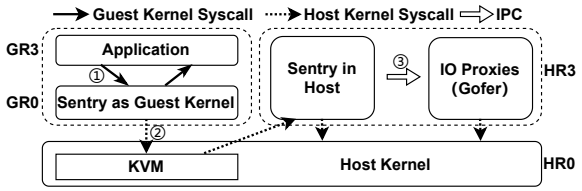

Figure 4. Main code paths of gVisor.

As shown in Figure 4, there are three main paths in gVisor to handle syscalls from user applications: ① The *guest kernel syscall path*: gVisor intercepts and implements syscalls for the application and acts as the guest kernel (**L1 syscall**). ② The *host kernel syscall path*: Sentry's construction relies on a set of syscall interfaces provided by the host kernel (**L2 syscall**), such as process and memory management, network drivers, and low-risk file I/O [2]. ③ The *file I/O proxy path*: for sensitive file operations such as *open()* and *stat()*, gVisor communicates with Gofer for I/O proxy, mitigating risks associated with direct access to the host file system (file I/O).

2.3 Performance Bottlenecks: An Industry View

In this section, we report the performance data of a Java-based E-Commerce container instance. Table 3 shows real consumer requests (100–800 TPS), while Table 4, Table 5, and Table 7 show simulated user requests (220 TPS).

Characteristics-2: Performance in real-world scenarios. We deploy gVisor, Kata, and runc in our production environment for comparative performance analysis. As shown in Table 3, gVisor's CPU usage increases by 31.23% during normal operations and 52.47% under peak workloads, while Kata increases by 5.16% and 19.33%, respectively.

Table 3. CPU Utilization and Response Time (RT) under production workloads.

Runtime	Normal		Peak	
	CPU Usage	RT (*ms*)	CPU Usage	RT (*ms*)
runc	5.79%	17.33	36.33%	22.03
Kata	10.95%	18.32	55.66%	23.89
gVisor	37.02%	19.43	88.80%	75.45

[2]We refer to file-related syscalls that directly call the host from the Sentry as low-risk file I/O. See Appendix A for details.

Characteristics-3: Application's syscalls. Table 4 provides a characterization of the syscall behavior of the application. We measure the number of syscalls generated by applications across different runtimes. Both runtimes exhibited similar patterns, with **I/O-related syscalls (file and network) comprising** 51.79% **and** 56.87%**, respectively**. These results align with prior studies [66, 103, 109].

Table 4. The number of syscalls (in K/s) categorized by kernel subsystem. The taxonomy and details of each syscall are provided in Appendix B.

Runtime	Total	File	Network	Mem.	Sync	Misc
runc	200.48	47.85	56.34	0.45	77.46	18.38
gVisor	173.11	48.34	50.11	0.80	54.45	19.13

Implication-3: *Cloud-native applications exhibit an **I/O-intensive** pattern due to high RPC and logging demands.*

Characteristics-4: gVisor's VMExit. Table 5(a) presents the breakdown of VM exit reasons for gVisor, which include four main categories: EXTINT (External Interrupt), CPUID (CPU Identification), VMCALL (Hypervisor Call), and EPTVIO (EPT Violation). Notably, VMCALL represents the highest proportion at 65.55%. Additionally, Table 5(b) details the classification of VMCALLs.

Table 5. The number of VMExit reasons and VMCALLs for gVisor per second. The taxonomy and details of each vmcall are provided in Appendix C.

VMExit Reasons	EXTINT	CPUID	VMCALL	EPTVIO
Counts (/s)	23,411	40	48,729	2,153

(a) The number of VMExit reasons.

Total	File	Network	Mem.	Sync	Misc
48,729	6,895	8	58	36,634	5,111

(b) The number of vmcalls categorized by kernel subsystem.

Implication-4: *gVisor enables the guest kernel to allocate and reclaim resources from the host kernel as a process, providing flexibility but incurring **frequent vmcalls**.*

Characteristics-5: gVisor's performance bottlenecks. To analyze gVisor's performance degradation, we use micro-benchmarks to quantitatively assess the cost of operations along different paths in Table 6. For *L1 syscall*, we test the *getpid()* syscall from the container application. For *L2 syscall*, we benchmark the Sentry kernel's invocation of the host kernel's *getpid()* function and compared it to direct calls in runc. We also evaluate network I/O performance by measuring ICMP, TCP, and UDP latency during idle states and assess file I/O performance via *open()* as a representation.

Table 6. The average round-trip latency for different code paths (*us*). *getpid()* is designed to evaluate the performance of the critical path where a ring transition occurs.

Type	Operations	runc	Kata	gVisor
L1 syscall	*getpid()*	N/A	0.345	0.849
L2 syscall	*getpid()*	0.266	N/A	6
Network I/O	ICMP	130	140	250
	TCP	420	470	640
	UDP	140	200	410
File I/O	*open()*	0.611	0.665	2.790

Implication-5: *Syscall forwarding (L1 & L2), Network I/O, and File I/O represent significant performance bottlenecks.*

Characteristics-6: Micro-architecture metrics. Microarchitecture analysis reveals significant differences between gVisor and runc in CPI, L1 and L2 cache misses, and TLB misses. As shown in Table 7, under the same workload, gVisor containers executed more instructions but exhibited lower microarchitecture efficiency, as evidenced by higher cache and TLB misses.

Table 7. The micro-architecture metrics. MPI stands for misses per instruction.

Metrics	runc	gVisor
CPI	1.51	1.65 (↑ 9.27%)
L1D MPI	2.02e-02	2.33e-02 (↑ 15.35%)
L1I MPI	4.32e-02	4.38e-02 (↑ 1.39%)
L2 MPI	2.01e-02	2.31e-02 (↑ 14.93%)
ITLB MPI	2.73e-06	1.59e-05 (↑ 482.4%)
DTLB load MPI	2.56e-03	2.75e-03 (↑ 7.42%)
DTLB store MPI	2.68e-05	7.97e-05 (↑ 197.39%)

Implication-6: *Process virtualization solutions like gVisor execute more instructions and exhibit lower microarchitectural metrics (higher CPI, Cache and TLB miss rates).*

2.4 Challenges and Goals

2.4.1 Challenges. As shown in Table 6, gVisor has a significant performance loss compared to microVM-based security containers and traditional containers. Specifically, we identified three performance bottlenecks and challenges.

Challenge-1: Syscall Service Overhead. For **L1 syscall**, the overhead arises from its Go runtime-based implementation, which includes the timer, scheduler, channel, and heap allocator. While this reduces kernel code and ensures memory safety, it introduces non-negligible overhead [25]. For example, the *getpid()* syscall takes 849*ns*, compared to Kata's 345*ns*. The Biscuit project [25] shows that a POSIX kernel in Go avoids most real-world bugs but is 5%–15% slower than C. For **L2 syscall**, issuing a host syscall on the KVM platform under x86_64 architecture involves switching from GR0 (Guest Ring 0) to HR0 (Host Ring 0), then to HR3 (Host Ring 3). This complex path (GR0->HR0->HR3->HR0) incurs significant overhead, approximately 20 times that of native host syscalls (0.266*us* for runc vs. 6*us* for gVisor).

Challenge-2: Network I/O Overhead. Cloud-native applications are sensitive to network performance. gVisor implements a user-mode network stack and sends/receives packets via the host Linux kernel's raw socket. As shown in Table 6, gVisor's latency for ICMP, TCP, and UDP operations is 1.52×-2.92× (1.36×-1.79×) higher than runc (Kata), failing to meet microservice workloads' network performance requirements. Existing device pass-through approaches improve network performance, but often require application rewriting, breaking compatibility, or lacking resource elasticity due to busy-spinning cores and pinned memory (e.g., mTCP [48], IX [22], Arrakis [85], eRPC [24]). Junction [30] proposes a libOS-based kernel bypass system but fails to meet our isolation and compatibility requirements (discussed in §2.1).

Challenge-3: Filesystem I/O Overhead. gVisor proxies file access through Gofer for isolation, but this enhances security at the cost of additional overhead, significantly harming performance. In the I/O proxy path, gVisor's *open()* operation is 4.56× slower than runc and 4.19× slower than Kata, making it a key bottleneck that degrades gVisor's overall performance.

2.4.2 Goals. To resolve these challenges, we present SKernel, a secure container that reconciles the elasticity-performance trade-off while maintaining compatibility and hardware-enforced isolation. The key of SKernel is its *split-kernel* architecture, which decouples resource management (host-coordinated) from performance-critical I/O operations (guest-local). SKernel adopts a secure-container model (virtualization + independent guest kernel). We implement SKernel based on gVisor due to its lightweight and elasticity. In contrast, MicroVM has a larger codebase, which complicates the implementation of certain optimizations. SKernel outperforms microVM-based systems through a suite of careful optimizations (as detailed in §4 and §5).

3 SKernel Overview

As shown in Figure 5, SKernel comprises three key components: **SKernel-R** (Resource Kernel), **SKernel-D** (Data Kernel), and a lightweight hypervisor (**SKernel-V**).

- **SKernel-R** is built on a minimal set of *resource interfaces* (listed in §5) to interact with the host kernel and is responsible for fine-grained resource management (e.g., CPU and memory), ensuring system-wide elasticity.
- **SKernel-D** handles I/O operations. For networking, it implements a device passthrough-based network stack (§4.2). Unlike passing network device directly to applications [3, 22, 30], SKernel passes it to SKernel-D, ensuring compatibility (e.g., multi-process, multi-threading, multi-language) and resource flexibility (e.g., spinning cores, pinned memory, application rewriting). It also supports a hybrid thread model and adaptable I/O notifications for

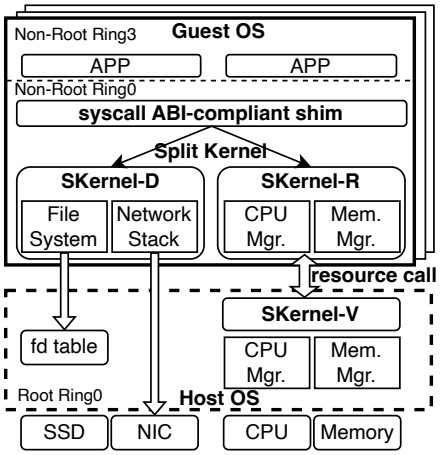

Figure 5. SKernel architecture.

cloud-native demands. For file-related I/O (§4.3), SKernel-D constructs a container file system based on shared file descriptors (instead of sharing a directory in gVisor), enhancing performance and security by eliminating the IPC-based Gofer process and reducing host kernel interactions.

- **SKernel-V** is an optimized lightweight hypervisor designed for containerized cloud-native workloads. It provides virtualization support for SKernel and serves as an efficient channel for guest-to-host resource calls (§4.1.2).

4 Detailed Design

4.1 Fast Syscall Forwarding (Challenge-1)

SKernel introduces *syscall pre-routing* and *syscall bridge* to reduce the latency of gVisor's dual syscall costs, respectively.

4.1.1 Syscall Pre-Routing (L1). *L1 syscall overhead* arises from Sentry's Go-based syscall execution (e.g., stack management, goroutine), which incurs a 3.19× latency penalty compared to native Linux. A key insight is that while optimizing all syscalls is impractical, prioritizing performance-critical ones via a *syscall fastpath* [59] can mitigate this overhead. SKernel introduces an *ABI-compliant syscall shim* to split and accelerate datapath-related syscalls (e.g., I/O operations). This shim uses a trampoline to dynamically route syscalls: based on thread context and file descriptor (fd) type, it either follows the fastpath (via C-style calling conventions, akin to *cgocall*) or the default Sentry path. Fastpath syscalls bypass SKernel-R and are directly handled by SKernel-D's syscall handler, eliminating Go runtime overhead. This design retains Sentry's isolation guarantees while enabling modular extensions (e.g., custom handlers) for future optimizations.

4.1.2 Syscall Bridge (L2). *L2 syscall overhead* arises from the lengthy guest-to-host kernel call path (GR0→ HR0→ HR3→ HR0, as shown in Figure 4), which incurs frequent context switches. SKernel addresses this by bypassing the intermediate HR3 step: instead of redirecting syscalls to user space, SKernel-V intercepts *vmcall* instructions and directly executes them in the host kernel. When SKernel-R issues a Resource Interface call, the *vmcall* instruction triggers a VM-exit to SKernel-V. SKernel-V intercepts the *vmcall* at HR0, parses the invocation context from the vCPU registers, and constructs the corresponding syscall context. It then emulates a logical HR3→HR0 syscall by directly invoking the host kernel interface via a function call, while preparing the host-side call stack and handling the return values. This reduces the call path to GR0→HR0, eliminating redundant context switches while retaining compatibility with existing hypervisor interfaces.

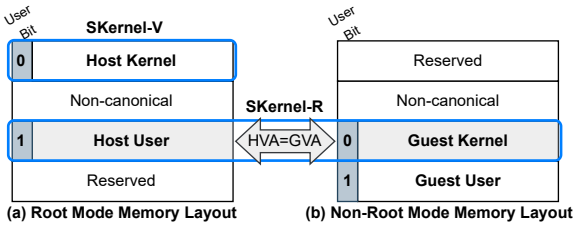

Figure 6. Memory layout.

The parameters may pass pointers in their address space that the host kernel needs to dereference. To facilitate this interaction across two distinct address spaces, SKernel-V maps a customized memory layout. As depicted in Figure 6, it treats each instance of SKernel-R as a regular process. The GVA (Guest Virtual Address) of SKernel-R section in VMX non-root mode corresponds to the HVA (Host Virtual Address) of the host user section. Additionally, SKernel manages separate address spaces for different application processes.

To ensure security, SKernel-V acts as a trusted guardian at the syscall boundary. Its small codebase (4.4K LoC vs. KVM's 52K LoC) reduces the TCB and attack surface. SKernel-V audits all guest-initiated syscalls using a `seccomp` filter to validate types and parameters, and further restricts memory access so that only predefined regions (e.g., DMA buffers) are accessible, with `ioctl()` limited to DMA management. These measures collectively mitigate guest-to-host exploits, including code-reuse attacks.

4.2 Network for Cloud-Native (Challenge-2)

SKernel provides a high-performance container network stack for cloud-native workloads. To address compatibility and isolation issues in prior work [22, 24, 48, 75, 80, 85, 110] discussed in §2.4.1, *SKernel passes network devices to* SKernel-D *instead of* applications*, maintains a gate between the app and guest kernel, and bypasses the host kernel (as shown in Figure 5).* Device pass-through in virtualized environments introduces challenges for flexibility and performance.

Challenge-1: Obstacle to memory overcommitment. Memory overcommitment enables cloud services to maximize resource utilization by allocating memory beyond physical limits, improving efficiency and reducing costs. However,

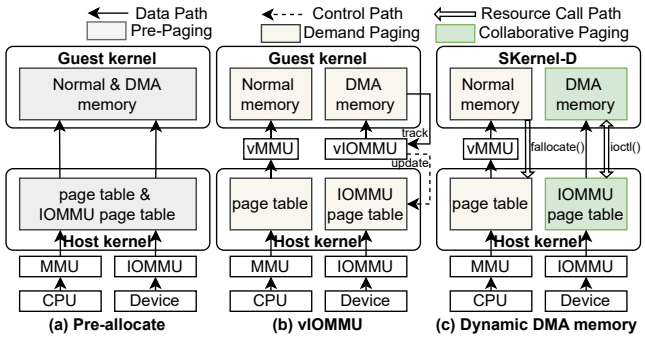

Figure 7. DMA Memory Management. vMMU refers to the hypervisor-maintained two-stage address translation, e.g., EPT [99].

it fails when device pass-through is enabled. As shown in Figure 7(a), IOMMU lacks IOPF support and must pre-pin all guest memory (Normal + DMA) because the host cannot track which parts are used for DMA. To prevent DMA failures [5, 104], memory overcommitment is disabled. Figure 7(b) shows vIOMMU approaches: vIOMMU [13] and coIOMMU [97] avoid DMA failures by tracking guest DMA memory usage and updating the I/O page table of the physical IOMMU, but this introduces performance overhead. VPRI [35] offers more efficient IOPF support with special DPU requirements. V-Probe [104] addresses performance issues but uses intrusive methods to detect guest memory usage, complicating deployment.

Challenge-2: Inefficient I/O Handling. I/O handling challenges involve two aspects: thread model and notification. First, optimizing the I/O thread model is critical yet complex. Two main approaches exist: the *inline model* (e.g., LUNA [111]) integrates stack and driver functions into task threads to improve cache locality but sacrifices batching opportunities. In contrast, the *look-aside model* (e.g., mTCP [48], Shinjuku [52]) isolates these functions into dedicated I/O threads via message queues, introducing additional tail latency. Second, the traditional interrupt-based notification mechanism frequently wakes the active vCPU to handle interrupts, causing significant latency when waking a suspended vCPU. This increases application response time and lowers system efficiency.

SKernel introduces *Dynamic DMA Memory* and *Optimized I/O Handling* mechanisms to address these two challenges.

4.2.1 Dynamic DMA Memory. As shown in Figure 7 (c), SKernel implements a dynamic DMA memory mechanism through collaborative paging. Within the memory regions allocated to SKernel (at low addresses, as in Figure 6), a dedicated DMA region is used for dynamic memory allocation. The DMA and non-DMA regions are isolated and do not interfere. Through SKernel-R interfaces, the guest kernel can dynamically allocate or reclaim memory using host APIs like

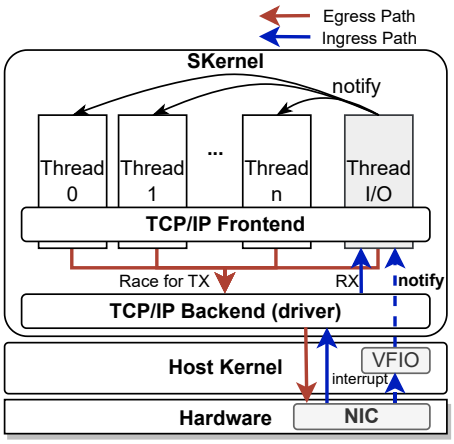

Figure 8. I/O Handling in SKernel. (a) In the egress path, I/O threads and regular threads can send packets, with regular threads ensuring fast delivery when I/O threads are in sleep mode. (b) In the ingress path, interrupts are handled exclusively by the I/O thread, which dynamically adapts between interrupt and polling modes based on traffic patterns.

mmap()/munmap() with `ioctl()` to update I/O page tables, avoiding IOPF and maximizing flexibility. Leveraging the predictable DMA memory needs of cloud-native workloads, SKernel efficiently handles traffic bursts by scaling memory resources on demand and reclaiming them afterward. This dynamic approach optimizes performance and resource utilization, ensuring agility in cloud-native environments.

4.2.2 Optimized I/O Handling. As shown in Figure 8, SKernel optimizes the I/O thread model and notification mechanism, improving performance in fluctuating scenarios.
Hybrid Thread Model: SKernel integrates both ***inline model*** and ***look-aside model*** to optimize batching efficiency and minimize latency. Application threads directly interact with the protocol stack front-end for seamless operations. In scenarios where the I/O thread is in interruptible sleep mode during packet transmission, application threads handle back-end processing to ensure swift packet delivery. Meanwhile, packet reception is handled by a dedicated I/O thread. Once the driver and protocol stack processing are complete, app threads are notified via channels.
Adaptable I/O Notification: SKernel adopts an event-driven I/O notification mechanism combined with adaptive polling techniques, moving away from traditional per-vCPU interrupt-based notification. It intelligently switches between ***interrupt mode*** and ***polling mode***, dynamically adapting to traffic patterns. Specifically, network tasks are assigned to a dedicated I/O thread, minimizing performance disruptions caused by unpredictable packet arrivals and avoiding the overhead of thread synchronization. When high-throughput network traffic is detected, the I/O thread switches into polling mode to ensure low-latency and efficient packet

processing. Conversely, if no packets are received after 5 consecutive polling iterations, it reverts to interrupt mode to conserve CPU cycles. This adaptive switching mechanism achieves a balanced trade-off between performance and efficiency in virtualized network processing.

4.3 FD-based Container Filesystem (Challenge-3)

Existing container runtimes typically use a shared file system to access directory trees on the host (e.g., rootfs, user code, layers) [29, 42, 71]. This requires unpacking user code on the host, introducing security risks. Additionally, binding mounted directories often relies on a proxy, leading to performance overhead at startup and runtime. In gVisor, Gofer [40] serves as a file proxy between the Sentry kernel [41] and the host kernel (details in Appendix A).

SKernel's file system design enhances security and performance by removing the IPC-based Gofer process and minimizing interactions with the host kernel (only 2 fds instead of an entire directory). It incorporates a read-only file system (ROFS) and a read-write file system (RWFS): (1) ROFS is based on EROFS [32, 106], a high-performance read-only file system for efficient compression and fast read access in Linux. It directly mounts raw image files from the host kernel, limiting interactions to a few file descriptors and reducing overhead and potential attack surfaces. (2) RWFS provides a tmpfs-like structure within SKernel, leveraging the host kernel's memory reclamation and write-back mechanisms to flush least-recently used pages into a regular file on the host, serving as ephemeral storage for guest applications.

Upon initialization, the rawfile image's file descriptor (fd) on the host is sent to SKernel. SKernel then uses `mmap()` to map the image content and mounts it into the guest kernel's directory via EROFS and TMPFS drivers. This reduces runtime overhead and the number of host fds held, while preventing indirect access to other host files through Gofer.

5 Implementation

SKernel is POSIX-compatible and inherits the full set of system call support from gVisor to support unmodified Linux binaries[3]. **SKernel-R**, derived from gVisor's Sentry [41] and comprising 42,991 lines of Go and 1,323 lines of Rust, implements guest-side syscall handling, including memory management, task scheduling, and interrupt handling. It coordinates resource allocation with the host kernel through *Resource Interfaces*, while delegating performance-critical I/O operations to *SKernel-D*. **SKernel-D** consists 18,728 lines of C code and comprises two components: the network stack and the filesystem. We implement a userspace TCP/IP stack based on TLDK[2] and use DPDK[3] as the NIC driver. We also integrate the driver support filesystem for EROFS and TMPFS in SKernel-D.

[3]The source code has been contributed to the gVisor community, and all patches are organized and available at https://github.com/antgroup/SKernel.

SKernel-V is a lightweight hypervisor designed for cloud-native applications. It is implemented as a kernel module and eliminates unnecessary devices(e.g., BIOS, PIT, PIC, IOAPIC), and CPU instruction emulation that not required for container workloads. SKernel-V offers two main functionalities. Firstly, it provides resource call capabilities for SKernel. Secondly, it has undergone microarchitecture optimizations, such as transparent huge pages [27] and TLB Sharing.

Resource Interfaces. SKernel retains only ***10*** resource interfaces for resource management, categorized into vCPU, memory, and I/O. First, SKernel requires 6 system calls to manage vCPU. SKernel utilizes *tgkill()* and *futex()* to implement the vCPU notification mechanisms, and uses *epoll_pwait()* to waite for I/O events. This approach offers the advantage of simplifying SKernel-V's implementation. It uses the *clone()* function to effectively generate a new vCPU, thus addressing the challenge of vCPU hot-plugging complexity within the QEMU/KVM architecture [65]. Finally, *exit()* and *exit_group()* are utilized to destroy a vCPU and terminate the entire container, respectively.

Second, in order to improve the utilization of host memory resources and guest memory overcommitment, SKernel provides an efficient *fallocate()* interface for SKernel to directly and effectively notify the host about available free memory pages. This surpasses existing mechanisms such as memory ballooning [20, 101], hyperupcall [14], and V-Probe [104].

Third, SKernel relies on the *ioctl()* function to program the IOMMU page table, thus enabling efficient memory management in direct I/O. As a result, there is no need to pin the DMA memory statically in advance or pin the entire guest memory. Notice that we only allow SKernel to use *ioctl()* for managing its DMA memory region. Other functions are not allowed, and the parameters are strictly checked to avoid *ioctl()* related security issues [107]. SKernel also relies on *mmap()* and *munmap()* to support file system design.

TLB Sharing. SKernel employs an optimization, **per-process PCID and per-instance VPID mechanism**, to reduce the frequency of TLB flush and improve the utilization of TLB entries. Specifically, SKernel assigns a unique VPID to each container instance, ensuring that all virtual logical processors (vLPs) within an instance share the same VPID (instead of separate VPIDs). This has the advantage that TLB data can be used (if present) when multiple threads of a process are migrated across different logical/physical processors.

Furthermore, each guest process is allocated a unique PCID. When an unmap event occurs in a guest process, SKernel does not flush the associated TLBs immediately; instead, it assigns a new PCID from the list of "available PCIDs" for that process. The old PCID is moved to the "used PCIDs" list and is not reused until it has been reclaimed. If no VPIDs are available on the host, SKernel reclaims all VPIDs in the list of "used VPIDs" and repeats the process. This can prevent unnecessary TLB flushes.

6 Evaluation

6.1 Methodology

Experimental setup. For end-to-end evaluation (§6.2), performance breakdown (§6.3), and production evaluation (§6.5), we all use our new production platform for testing. Each server in the new platform is equipped with an Intel Xeon 8369B 2.90GHz 64-core CPU, 1TB of RAM, and a MOC 2.0 NIC — a network interface card model offered by the cloud provider that supports SR-IOV and provides multiple virtual functions (VFs). Micro-benchmark experiments in §6.4 run on our old platform with each server's configuration of an Intel Xeon E5-2682 2.50GHz 32-core CPU, and 256GB of RAM and MOC 1.5 NIC[4]. All platforms run Linux 5.10 with KPTI and PCID enabled.

Our choice of baselines follows three criteria: (i) production-proven systems that are widely deployed in large-scale cloud environments, (ii) open-source projects with active community support, and (iii) the ability to run on the same hardware configurations used by SKernel. We compare **SKernel** with traditional *process-based* container **runc** (1.0.0-rc10) [44], *microVM-based* secure container **Kata** (v3.4.0) [29] and *process virtualization-based* **gVisor** (tag-release-20240501.0) [42]. While unikernel solutions [64] have promising performance potential, they currently lack the deployment maturity and compatibility required for production-scale evaluation in our environment (see §2.1). All tested container instances use 8 CPUs, 16GB of memory and 10GB ephemeral storage [63]. All experiments were conducted with NUMA enabled and each container instance's cpuset is allocated to the same NUMA node, which reflects the configuration of real-world production environments.

For **runc**, we disable RPS (Receive Packet Steering [6]) and RFS (Receive Flow Steering [38]) because the server's NIC already supports SR-IOV with multiple hardware queues, while software-based steering provides no additional benefit and can even degrade performance. This configuration ensures fairness, as disabling RPS/RFS improves runc's throughput; enabling them would instead amplify SKernel's advantage. **Kata** and **gVisor** are evaluated under the same host configuration. Kata incorporates VFIO pass-through devices [7] to enhance network performance and employs virtio-fs [100] for file operations. gVisor uses Gofer [40] for file system operations and its own netstack [37] for networking.

6.2 End-to-end Application Performance

We continue the evaluation of SKernel using four representative real-world applications with distinct characteristics.

1. *Sofaload* is a benchmark developed based on h2load [79], specifically designed for evaluating microservices. The server container comprises a sidecar network proxy mesh [16] implemented in Go and a Java application based on SOFABoot [92]. The client's request is first sent to the sidecar, which then forwards it to the Java application for processing and response.
2. *Redis* is an in-memory database (version 7.2.3) [87]. We use redis-benchmark utility [88] for the evaluation. The value size for SET/GET is fixed at 1000 bytes, with a pipeline request number of 16 and a total of 10 million requests. The number of parallel connections is 300. We take the performance metric of PING_BULK(P_B), SET, GET, and INCR.
3. *Nginx (version 1.20.1)* is a web server [4]. The wrk benchmarking tool [102] is used to measure throughput in different scenarios (Non-Url refers to 404 Not Found requests). The test duration is 30 seconds, with 4 threads and 1,000 HTTP connections.
4. *MySQL (version 5.7.40)* is a widely-used relational database [82]. To evaluate system performance, we employ sysbench [60] to measure transaction and query metrics. We configure the test with four tables, each containing 100,000 rows of data. The number of concurrent threads is set to 512, and the test runs for 300 seconds.

These applications are chosen to represent *CPU-intensive*, *memory-intensive*, and *file I/O-intensive* workloads, respectively. The server and the client are deployed on different hosts for performance isolation, and they all rely on efficient networks (*network I/O-intensive*), which also reflects the real deployment situation of the microservice architecture.

The *sofaload* testcase closely mirrors our production environment. This case primarily evaluates the efficiency of message serialization and RPC communication between the client and server. We test the maximum achievable request throughput per second for a fixed number of clients (64) making requests to a single server. As shown in Figure 9 (a), we evaluate the throughput of various systems, and SKernel achieves the best performance, which is 1.68×, 1.23×, and 1.18× faster than gVisor, Kata, and runc respectively. This validates the effectiveness of its design, especially the network-related optimizations. gVisor shows the poorest performance due to the three bottlenecks discussed in Section 2.4.1, while Kata demonstrates slightly lower performance than runc due to virtualization overhead.

Figure 9(b) shows the performance of Redis in each container. SKernel performs the best, with performance gains of 1.81×–2.34× over gVisor for different operations (1.13×–1.27× compared to Kata, and 1.07×–1.18× compared to runc).

Figure 9(c) shows the throughput of Nginx with different web page sizes and varying concurrent requests. SKernel achieves a speedup of 3.42×–4.04× in throughput compared to gVisor (1.69×–1.98× compared to Kata, and 1.40× – 1.67× compared to runc).

Figure 9(d) presents the MySQL throughput. SKernel achieves a 4.49× speedup in both transaction and query

[4]We use the old platform for micro benchmarks instead of the new platform because the new production platform does not allow us to freely change some critical settings, such as KVM settings.

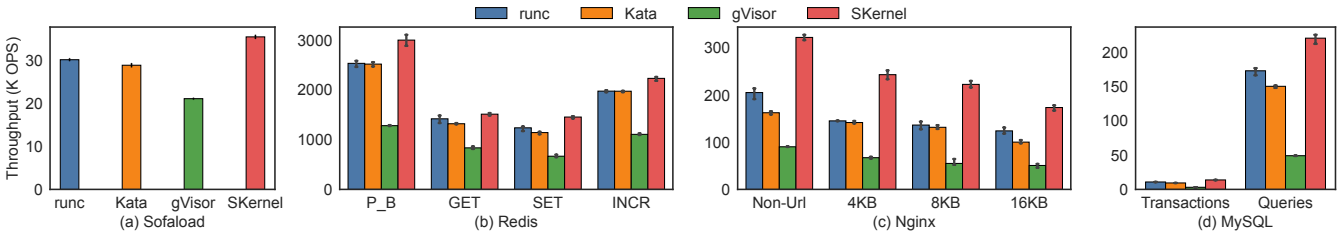

Figure 9. Performance comparison of typical applications.

performance compared to gVisor, 1.47× over Kata, and 1.28× compared to runc.

SKernel consistently demonstrates superior performance in evaluations using representative applications—Nginx, Redis, and MySQL—primarily due to its high-performance network stack, optimized syscall path, and filesystem optimizations. The performance advantage is most pronounced in the Nginx and MySQL benchmarks, where SKernel outperforms gVisor by enhancing file I/O through the introduction of an FD-based container filesystem (§4.3) that improves upon gVisor's Gofer design.

6.3 Optimization Breakdown

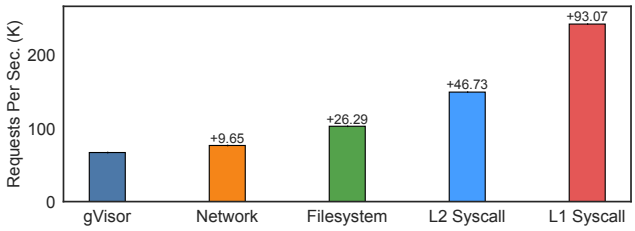

Figure 10. Performance improvement breakdown.

To delve deeper into the contribution of each optimization, we evaluate and analyze the performance by enabling different optimizations in SKernel step by step. We use Nginx as a test case (connections = 10000, web page size = 4 KB) and gVisor as a baseline.

In Figure 10, the *network* optimization refers to the high-performance network stack in SKernel, which improves performance notably by +9.65K RPS. The custom file system in SKernel eliminates the Gofer file proxy overhead, which achieves +26.29K RPS (*filesystem*). To further reduce overhead, we optimized the call path between the guest and host kernels and implemented a lightweight hypervisor tailored for containerized workloads, which results in a +46.73K RPS gain (*L2 Syscall*). Finally, addressing gVisor's syscall handling bottleneck, we implement a fast path bypassing the Sentry's syscall handler (*L1 Syscall*), which achieves +93.07K RPS. Each optimization builds on the previous ones, amplifying the overall performance improvements.

6.4 Micro Benchmark Evaluation

We compare SKernel's average system calls operation, network I/O operation, and file I/O operation performance with that of other systems in microbenchmarks. We use *getpid()*, as described in §2.4.1, to test the L1 syscall and the L2 syscall. To evaluate network I/O operation, the ping utility from iputils [45] is used to evaluate ICMP latency, while TCP latency is measured by simulating an Nginx 404 error response scenario with http-ping as the client. For precise UDP latency measurements, we develop a custom echo server using epoll, recv, and write functions. To evaluate file I/O operation, we use FIO [19] to test file operation performance of different systems with write-through disabled (`direct=false`). Each test is executed 10,000 times, and the average result is reported.

Table 8. Comparison of different systems' performance in microbenchmarks.

Type	Operations	runc	Kata	gVisor	SKernel
L1 syscall (us)	*getpid()*	N/A	0.345	0.849	0.320
L2 syscall (us)	*getpid()*	0.266	N/A	6	0.289
Network I/O (us)	ICMP	130us	140	250	110
	TCP	420	470	640	390
	UDP	140	200	410	190
File I/O ($MB/s, K$)	wr-bw	795	1293	359	1182
	wr-iops	194	316	87.7	302
	rd-bw	1725	1463	465	1700
	rd-iops	422	357	114	435

The results are presented in Table 8. For L1 syscalls, SKernel bypasses the gVisor syscall handler through the syscall fast path, reducing the time from request to processing. For L2 syscalls, SKernel directly invokes syscalls via vmcall, shortening the call path. Its performance is comparable to that of host applications directly calling the host kernel, with only an additional VM-exit. For network I/O operation, SKernel benefits from its high-performance net stack, surpassing runc, Kata, and gVisor in ICMP and TCP (UDP is slower than runc because it has not been integrated into the syscall fast path). For file I/O write operation, SKernel's RWFS writes to buffers similar to Kata, while achieving higher write performance compared to runc at the cost of durability. SKernel periodically flushes into the rawfile, Kata does not implement flush to disk[8]. For file I/O read operations, SKernel utilizes

ROFS, which only requires the host kernel to populate the EPT page table. In contrast, Kata's virtiofs necessitates additional communication with virtiofsd. The performance of SKernel is similar to that of runc and better than that of Kata.

6.5 Production Evaluation

We only compare runc and SKernel in production evaluation. Because our cloud previously used runc and has now migrated to SKernel, making runc the natural baseline, and other systems are not deployed in our production environment.

6.5.1 Performance. We compare the CPU usage of SKernel and runc under the same TPS requests, analyzing both typical daily traffic and peak burst traffic (shopping festivals).

(1) Daily Traffic. We capture the operation of an RPC services application for a whole day, focusing on its CPU utilization, as illustrated in Figure 11. SKernel consistently exhibits lower CPU usage than runc. On average, the CPU usages are 15.75% and 12.03% for runc and SKernel respectively, SKernel reduces the average CPU consumption by 3.72% under the same level workload. In the burst traffic mode, the performance will be even more remarkable. Under bursty workloads, SKernel achieves an improvement of 8.73% compared to runc. This is because SKernel can switch processing modes according to the changes in the workload. In bursty workloads, the adaptable I/O notification design of SKernel will switch from interrupt mode to polling mode.

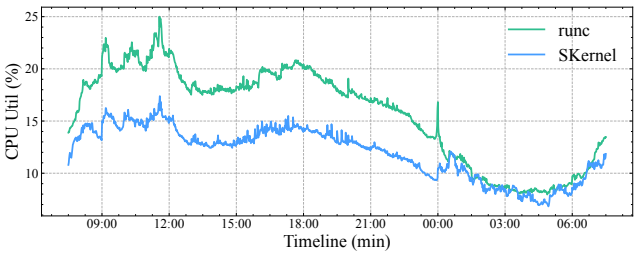

Figure 11. Comparison of CPU Utilization (lower is better). Data was collected from 7:30 AM to 7:30 AM the next day. Both systems have the same request level, with a peak at 11:30 AM (approximately 400 TPS) and a trough at 4:30 AM (approximately 10 TPS).

(2) Shopping Festival's Peak Traffic. Every shopping festival season, we purchase or rent additional machines in advance to keep CPU load at a healthy level (20%–40%) and meet the SLAs[39]. If deployed with Kata or gVisor, we need additional cost due to their performance degradation (discussed in §2.3). SKernel can match the performance of the traditional runc and provides a cost-effective solution. As shown in Table 9, we select four important, high-traffic applications from two data center regions for observation. These applications are Java-based online payment services with CPU- and I/O-intensive workloads, characterized by highly concurrent execution and bursty request distributions (i.e., a large volume of requests arriving within short time intervals). Each application is deployed within a secure container and utilizes sidecars to access distributed databases, log data, and connect to other RPC services. During peak hours of the day and under the same request load (about 612TPS), SKernel reduces CPU usage by 0.53%–3.99% compared to runc. These peak-hour results demonstrate that the SKernel architecture can effectively conserve CPU cycles and sustain a low utilization level even under intense workloads.

Table 9. The CPU usage of pods for different applications during peak requests in two regions. The same application pod in the same region will have the same workload magnitude. CPU Util measures the average CPU usage per region for each application using runc and SKernel pods.

Applications	Runtime	Region A		Region B	
		CPU Util	Pods	CPU Util	Pods
ttc	runc	26.65%	95	13.11%	51
	SKernel	23.09%	5	12.58%	50
pmtc	runc	21.36%	176	15.55%	131
	SKernel	20.50%	20	14.77%	69
cctb	runc	25.61%	205	17.93%	201
	SKernel	21.62%	46	16.09%	26
ccmc	runc	10.43%	89	11.43%	122
	SKernel	9.43%	27	8.92%	8

6.5.2 Elasticity. The elasticity of SKernel is primarily evident in three areas: CPU scalability, normal memory scalability, and dynamic DMA allocation. The first two aspects are inherited from gVisor, while the latter arises from the network stack design of SKernel. In this section, we evaluate the functionality of the DMA dynamic memory.

We evaluate how resource usage evolves over time for a container with dynamic resource requirements. We test Nginx and measure the size of DMA memory usage under varying request load. As shown in Figure 12, we conduct tests using wrk with 40 connections and 2 threads, requesting 128KB memory objects. At this configuration, the system achieves 7.7K Requests/sec with a DMA memory allocation of 30MB. When scaling to 5120 connections and 256 threads, the system reaches 14.3K Requests/sec while utilizing 542MB of DMA memory.

Figure 13 illustrates the total physical memory and DMA memory usage of each application in the production environment. The DMA memory for different applications is initialized at 8MB and dynamically scales to varying levels based on workload variations. Overall, in comparison to VM-based dedicated resource solutions (e.g., pinned memory), SKernel is capable of fulfilling the dynamic resource demands of fine-grained modular applications under real-world loads.

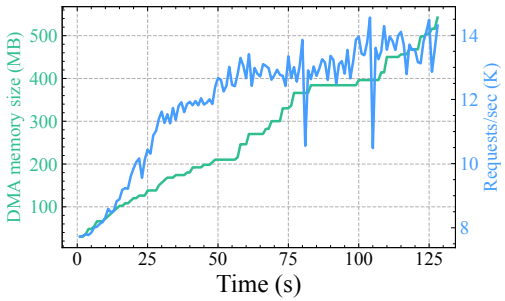

Figure 12. DMA memory usage under different loads.

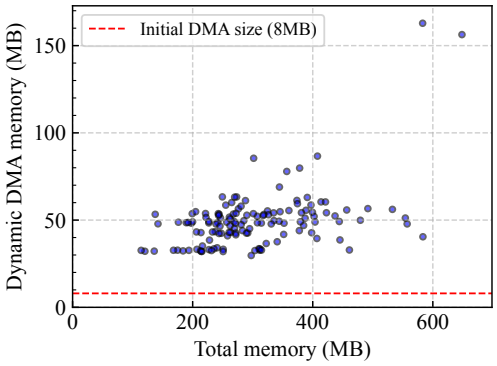

Figure 13. The RSS memory usage per application in the production environment.

6.6 Security Analysis

SKernel follows the threat model of existing secure containers [8, 29, 42, 71], where applications running within the container could be compromised or malicious [107, 108]. SKernel, SKernel-V, and the underlying host kernel are considered potential attack targets.

SKernel is built using memory-safe and type-safe languages like Go (inherited from Sentry) and Rust (used in the syscall fast path). It enhances security by eliminating shared host directories with guests, minimizing the attack surface from shared resources between containers and the host. SKernel-V utilizes hardware-assisted virtualization to isolate SKernel instances. SKernel-V only needs to handle 9 types of VM exits from non-root mode, with a compact codebase of 4,421 lines of code (LoC). This significantly reduces the attack surface compared to KVM [58], which handles 64 types of VM exits and has a codebase of 52,000 LoC.

For host kernel, we focus on potential security risks associated with Resource Interface and device pass-through:

Resource Interface: SKernel requests services from the host kernel via Resource Interface, with SKernel-V translating `vmcall` into `syscall`. These interfaces are secure and hardened against guest-to-host attacks. First, SKernel reduces security risks by restricting the number of allowed Resource

Table 10. Kernel crossings (per second) when running sofaload for 10s with an offered load of 32,000 RPS.

Runtime	Unique syscalls	Total syscalls (/s)	Unique VMEXITs	VMEXITs (/s)
runc	21	579,941	N/A	N/A
Kata	10	180	10	138,349
gVisor	28	522,296	6	147,103
SKernel	3	37,276	3	29,187

Interface calls, significantly fewer than in *microVM-based* secure containers [15, 30]. To demonstrate this, Table 10 report host kernel interactions through both system calls and VM exits. Compared to gVisor, SKernel lifts more I/O-related functionalities into the guest side instead of relying on the host, thus issuing fewer host interactions. Compared to Kata, SKernel's design avoids the extra VM exits overhead introduced by full hardware virtualization. For example, 72.99% of VM exits are MSR_WRITE and 23.79% are HLT in Kata, which do not exist under SKernel. Due to its reliance on the host's interface, the number of system calls has increased, but the types are smaller. Second, since SKernel-V is solely responsible for constructing the host kernel call stack and processing the returned results (Syscall Bridge in §4.1.2), there is no direct invocation path from the guest to the host kernel, ensuring strong isolation guarantees. It also inspects Resource Interface calls using seccomp-ebpf [54] to prevent malicious attempts to compromise the host kernel. From the host kernel's perspective, handling `syscalls` from Resource Interface and a user process from HR3 differs only in caller privilege level; both follow the same credential mechanisms [83] to ensure system security.

Device pass-through: SKernel directly assigns NIC devices to SKernel, allowing a single network card to be shared among multiple instances. Device memory isolation is ensured by configuring the host kernel's IOMMU page table, which SKernel can only modify via Resource Interface. These modifications are also validated by SKernel-V to maintain device memory isolation.

6.7 Lessons Learned and Experiences

Necessity of Application-Infra Isolation. Sidecars are critical for managing microservices. Current mainstream methods, such as Istio [46], utilize a **Per-Pod Sidecar** approach, where the sidecar and application share pod resources. This design, however, can lead to stability and security issues [94, 112]. An alternative is the **On-node Sidecar**, which fully decouples the sidecar from user applications along with eBPF-based traffic redirection(e.g., Canal [94], Ambient [93]). However, this approach is primarily effective in traditional containers and suffers from performance penalties when transferring app and sidecar traffic across VMs in secure containers. SKernel adopts a more practical solution by maintaining a Per-Pod Sidecar architecture while isolating the app from the sidecar through a secure kernel implemented based on a memory-safe language.

Making Pass-Through Practical for Containers. While existing device pass-through approaches [22, 31, 48, 52, 110]

map network queues to user space, enabling applications to communicate directly with the NIC and bypass kernel overhead, they often require code modifications, busy-spinning cores, and pinned memory, which can hinder feasibility in production environments. Recent advancements, such as Junction [30], leverage libOS as an intermediary layer, allowing the use of DPDK without necessitating code changes. However, this approach compromises isolation by requiring the sidecar and application packets to operate within the same process. Our work, in fact, makes device pass-through more practical within SKernel-D. Applications can achieve performance improvement without any modifications. Pinning a large amount of memory is not required. And container isolation is also preserved.

Feasibility of Userspace Kernel in Microservices. Achieving full Linux compatibility is challenging; however, developing a Linux ABI-compatible kernel is practical for microservice architectures. SKernel has fully implemented 164 syscalls and partially implemented 74, with no compatibility issues reported across nearly 2,000 applications and 100,000 pods. Recent work Loupe [67] proposes systematic methods to quantify Linux compatibility by analyzing syscall coverage and semantic gaps. Incorporating such approaches could provide a more rigorous assessment of SKernel 's compatibility guarantees in the future. Additionally, since emulating an entire machine is often unnecessary, we implement a *vmcall*-based lightweight hypervisor, without using KVM which is commonly used in microVM-based hypervisors. This allows us to simplify our setup by avoiding the complexities of platform emulation (such as IOAPIC and BIOS), resulting in a more efficient, process-based virtualization with reduced overhead.

7 Related Work

Besides systems in §2, we discuss other related work here.

Security-enhanced containers. *Rule-based security layers* [17, 47, 56] restrict the subset of kernel APIs available to containers, reducing the exposed attack surface but necessitating careful tuning of application-specific profiles. While they cannot provide the same isolation level as hardware-assisted virtualization, they can be used as a runtime inside a VM. Confidential containers based on TEEs or enclaves [9, 18, 51, 68, 70, 90] provide stronger protection but often require compromises in performance and flexibility.

Dataplane OSes. Systems such as Arrakis [85], IX [22], and Demikernel [110] pioneered kernel-bypass and data-plane specialization, leveraging direct device pass-through and user-space protocol stacks to minimize host kernel involvement and deliver high performance for network-intensive workloads. However, these designs often require substantial application or toolchain modifications and provide limited compatibility with unmodified Linux binaries, which restricts their deployment in multi-tenant cloud environments. Junction [30] and LUNA [111] take similar approaches by intercepting syscalls through customized toolchains, but this is not feasible in public cloud settings. In contrast, SKernel draws inspiration from data-plane OSes while making a different set of design trade-offs driven by production needs: it bypasses the host kernel for performance-critical I/O via SKernel-D, but coordinates CPU and memory management with the host through SKernel-R to preserve compatibility and resource flexibility. This split-kernel architecture enables strong isolation and full POSIX compatibility, offering a path toward practical deployment at scale.

Para-virtualization systems facilitate collaboration between the operating system and hypervisor to enhance functionality [72, 89], increase flexibility [33, 104], and optimize performance [14, 50, 55]. SKernel adopts a similar methodology but employs a holistic design from a ***guest-host kernel coordination*** perspective to run multi-tenant cloud services.

8 Conclusion

This paper presents SKernel, the first production-proven secure container that harmonizes performance and elasticity at scale. According to different resource characteristics, SKernel is designed with a split-kernel architecture and decouples the guest kernel into two components: SKernel-R (resource management via host collaboration) and SKernel-D (high-performance I/O operations without host involvement). To further enhance performance and elasticity, SKernel propose a fast syscall forwarding approach that minimizes context switch overheads, a high-performance network protocol stack suitable for cloud native workloads and a file descriptor-based container filesystem that enhances security, eliminates unnecessary indirection, and reduces costs. Our evaluations show that SKernel achieves 4.49× throughput compared to state-of-the-art solutions and supports elasticity and on-demand usage for volatile and bursty workloads. SKernel has been deployed in Ant Group's production environment at large scale and demonstrates robustness and cost-effectiveness.

9 Acknowledgments

We sincerely thank our shepherd Takahiro Shinagawa and the anonymous reviewers, whose reviews and suggestions greatly strengthened our work. This work was supported by Smart Gird-National Science and Technology Major Project (No.2024ZD0803000) and National Natural Science Foundation of China (No. 62302300). Correspondence to: Yu Chen (yuchen@tsinghua.edu.cn), Kang Chen (chenkang@tsinghua.edu.cn).

References

[1] 9p (protocol). https://en.wikipedia.org/wiki/9P_(protocol). Accessed: 2024-06-17.

[2] Archived-TLDK - fd.io. https://wiki.fd.io/view/TLDK. Accessed: 2024-06-17.

[3] Data plane development kit (dpdk). https://www.dpdk.org/. Accessed: 2024-06-17.

[4] Nginx. https://nginx.org/en/. Accessed June 13, 2024.

[5] Firecracker issue. https://github.com/firecracker-microvm/firecracker/issues/1179/, 2019.

[6] Red hat enterprise linux performance tuning guide: Networking configuration tools, 2024.

[7] VFIO: Virtual function i/o. https://docs.kernel.org/driver-api/vfio.html, 2024. Accessed: 2024-06-20.

[8] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA, February 2020. USENIX Association.

[9] Adil Ahmad, Botong Ou, Congyu Liu, Xiaokuan Zhang, and Pedro Fonseca. Veil: A protected services framework for confidential virtual machines. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS '23, page 378–393, New York, NY, USA, 2024. Association for Computing Machinery.

[10] Amazon Web Services. What is cloud native? - cloud native applications explained - aws. https://aws.amazon.com/what-is/cloud-native/?nc1=h_ls. Accessed: 2025-01-10.

[11] Amazon Web Services. What is service mesh? - service mesh explained - aws. https://aws.amazon.com/what-is/service-mesh/. Accessed: 2025-01-10.

[12] Amazon Web Services. Vertical pod autoscaler for amazon eks. https://docs.aws.amazon.com/eks/latest/userguide/vertical-pod-autoscaler.html, 2024. Accessed: 2025-05-13.

[13] Nadav Amit, Muli Ben-Yehuda, IBM Research, Dan Tsafrir, and Assaf Schuster. vIOMMU: Efficient IOMMU emulation. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*, Portland, OR, June 2011. USENIX Association.

[14] Nadav Amit and Michael Wei. The design and implementation of hyperupcalls. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 97–112, Boston, MA, July 2018. USENIX Association.

[15] Anjali, Tyler Caraza-Harter, and Michael M. Swift. Blending containers and virtual machines: a study of firecracker and gvisor. *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, 2020.

[16] Ant Group. MOSN (Modular Open Smart Network): The Cloud-Native Network Proxy Platform. https://github.com/mosn/mosn. Accessed: 2025-01-12.

[17] *AppArmor*. https://www.kernel.org/doc/html/v4.15/index.html.

[18] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, Dave Eyers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. SCONE: Secure linux containers with intel SGX. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 689–703, Savannah, GA, November 2016. USENIX Association.

[19] Jens Axboe. fio - flexible i/o tester rev. 3.36. *FIO's documentation!*, 2024.

[20] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. Xen and the art of virtualization. *ACM SIGOPS Operating Systems Review*, page 164–177, Dec 2003.

[21] Adam Belay, Andrea Bittau, Ali Mashtizadeh, David Terei, David Mazières, and Christos Kozyrakis. Dune: safe user-level access to privileged cpu features. *Operating Systems Design and Implementation,Operating Systems Design and Implementation*, Oct 2012.

[22] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 49–65, Broomfield, CO, October 2014. USENIX Association.

[23] Xiaohu Chai, Tianyu Zhou, Keyang Hu, Jianfeng Tan, Tiwei Bie, Anqi Shen, Dawei Shen, Qi Xing, Shun Song, Tongkai Yang, Le Gao, Feng Yu, Zhengyu He, Dong Du, Yubin Xia, Kang Chen, and Yu Chen. Fork in the road: reflections and optimizations for cold start latency in production serverless systems. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation*, OSDI '25, USA, 2025. USENIX Association.

[24] Youmin Chen, Youyou Lu, and Jiwu Shu. Scalable rdma rpc on reliable connection with efficient resource sharing. In *Proceedings of the Fourteenth EuroSys Conference 2019*, Mar 2019.

[25] Cody Cutler, M. Frans Kaashoek, and Robert T. Morris. The benefits and costs of writing a POSIX kernel in a high-level language. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 89–105, Carlsbad, CA, October 2018. USENIX Association.

[26] Docker. Docker: Accelerated container application development. https://www.docker.com. Accessed: 2025-01-10.

[27] Linux Kernel Documentation. Transparent huge pages. https://www.kernel.org/doc/html/next/admin-guide/mm/transhuge.html, 2023. Accessed: 2025-05-09.

[28] Uber Engineering. Uber's tech stack, part one: The foundation. https://eng.uber.com/tech-stack-part-one-foundation/, 2016. Accessed: 2025-05-15.

[29] The OpenStack Foundation. *Kata Containers Architecture*. https://github.com/kata-containers/documentation/blob/master/design/architecture.md.

[30] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Inigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. Making kernel bypass practical for the cloud with junction. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 55–73, Santa Clara, CA, April 2024. USENIX Association.

[31] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, OSDI'20, USA, 2020. USENIX Association.

[32] Xiang Gao, Mingkai Dong, Xie Miao, Wei Du, Chao Yu, and Haibo Chen. EROFS: A compression-friendly readonly file system for resource-scarce devices. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 149–162, Renton, WA, July 2019. USENIX Association.

[33] Stefano Garzarella. io_uring passthrough. Presented at the KVM Forum 2020, 2020. Available online: https://static.sched.com/hosted_files/kvmforum2020/9c/KVMForum_2020_io_uring_passthrough_Stefano_Garzarella.pdf.

[34] Google Cloud. Vertical pod autoscaler. https://cloud.google.com/kubernetes-engine/docs/concepts/verticalpodautoscaler, 2024. Accessed: 2025-05-13.

[35] Kaijie Guo, Dingji Li, Ben Luo, Yibin Shen, Kaihuan Peng, Ning Luo, Shengdong Dai, Chen Liang, Jianming Song, Hang Yang, Xiantao Zhang, and Zeyu Mi. Vpri: Efficient i/o page fault handling via software-hardware co-design for iaas clouds. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 541–557, New York, NY, USA, 2024. Association for Computing Machinery.

[36] gVisor. gvisor: Users. https://gvisor.dev/users/, 2023. Accessed: 2025-05-15.

[37] Gvisor Project Developers. Networking — gvisor user guide. https://gvisor.dev/docs/user_guide/networking/, 2024.

[38] Red Hat. *Red Hat Enterprise Linux 7 Performance Tuning Guide: Configuring Receive Flow Steering (RFS) Section*, 2024. Accessed June 13, 2024.

[39] IBM. What is a service level agreement (sla)? https://www.ibm.com/think/topics/service-level-agreement. Accessed: 2025-05-09.

[40] Google Inc. *gVisor Gofer*. https://pkg.go.dev/gvisor.dev/gvisor@v0.0.0-20240613080552-e18e311202b7/pkg/sentry/fsimpl/gofer.

[41] Google Inc. *gVisor Sentry*. https://pkg.go.dev/gvisor.dev/gvisor/pkg/sentry/syscalls.

[42] Google Inc. *What is gVisor?* https://gvisor.dev/docs/.

[43] InfoQ. Paypal's journey with spring boot. https://www.infoq.com/presentations/paypal-spring-boot/, 2020. Accessed: 2025-05-15.

[44] Opencontainers Initiative. runc: A cli tool for spawning and running containers. https://github.com/opencontainers/runc. Accessed: 2024-01-17.

[45] The iputils Contributors. iputils: The home of ping, tracepath, clockdiff and other network debugging tools. https://github.com/iputils/iputils, 2024. Accessed: 2024-04-05.

[46] Istio Authors. Istio: Connect, secure, control, and observe services, 2024. The official documentation website for the Istio service mesh.

[47] Trent Jaeger, Reiner Sailer, and Xiaolan Zhang. Analyzing integrity protection in the SELinux example policy. In *12th USENIX Security Symposium (USENIX Security 03)*, Washington, D.C., August 2003. USENIX Association.

[48] EunYoung Jeong, Shinae Wood, Muhammad Jamshed, Haewon Jeong, Sunghwan Ihm, Dongsu Han, and KyoungSoo Park. mTCP: a highly scalable user-level TCP stack for multicore systems. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 489–502, Seattle, WA, April 2014. USENIX Association.

[49] JetBrains. The state of developer ecosystem 2022: Microservices. https://www.jetbrains.com/lp/devecosystem-2022/microservices/, 2022. Accessed: 2025-05-15.

[50] Weiwei Jia, Jiyuan Zhang, Jianchen Shan, and Xiaoning Ding. Making dynamic page coalescing effective on virtualized clouds. In *Proceedings of the Eighteenth European Conference on Computer Systems*, EuroSys '23, page 298–313, New York, NY, USA, 2023. Association for Computing Machinery.

[51] Yuekai Jia, Shuang Liu, Wenhao Wang, Yu Chen, Zhengde Zhai, Shoumeng Yan, and Zhengyu He. HyperEnclave: An open and cross-platform trusted execution environment. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 437–454, Carlsbad, CA, July 2022. USENIX Association.

[52] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for μsecond-scale tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 345–360, Boston, MA, February 2019. USENIX Association.

[53] Kata Containers Community. vcpu handling in runtime-rs. https://github.com/kata-containers/kata-containers/blob/main/docs/design/vcpu-handling-runtime-go.md, 2023. Accessed: 2025-05-13.

[54] Michael Kerrisk. Understanding user namespaces. https://lwn.net/Articles/857228/, 2021. Accessed: 2023-04-15.

[55] Ozgur Kilic, Spoorti Doddamani, Aprameya Bhat, Hardik Bagdi, and Kartik Gopalan. Overcoming virtualization overheads for large-vcpu virtual machines. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Sep 2018.

[56] Taesoo Kim and Nickolai Zeldovich. Practical and effective sandboxing for non-root users. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 139–144, San Jose, CA, June 2013. USENIX Association.

[57] Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov. OSv—Optimizing the operating system for virtual machines. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 61–72, Philadelphia, PA, June 2014. USENIX Association.

[58] Avi Kivity and Anthony Liguori. kvm : the linux virtual machine monitor. 2007.

[59] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. sel4: formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, SOSP '09, page 207–220, New York, NY, USA, 2009. Association for Computing Machinery.

[60] Alexey Kopytov. sysbench: Scriptable database and system performance benchmark. https://github.com/akopytov/sysbench. Accessed: 2025-09-13.

[61] Kubernetes. Production-grade container orchestration. https://kubernetes.io. Accessed: 2025-01-10.

[62] Kubernetes. Pods. https://kubernetes.io/docs/concepts/workloads/pods/, 2024. Accessed: 2025-01-08.

[63] Kubernetes Community. Ephemeral volumes. https://kubernetes.io/docs/concepts/storage/ephemeral-volumes/, 2024.

[64] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gaulthier Gain, Cyril Soldani, Costin Lupu, Ştefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. Unikraft: fast, specialized unikernels the easy way. In *Proceedings of the Sixteenth European Conference on Computer Systems*, EuroSys '21, page 376–394, New York, NY, USA, 2021. Association for Computing Machinery.

[65] Amit Kumar and Salil Mehta. Challenges revisited in supporting virt cpu hotplug. https://lore.kernel.org/all/82deccdf-ad02-4c7f-bcb6-20cfd2723c17@redhat.com/T/, 2023.

[66] Rakesh Kumar and B Thangaraju. Performance analysis between runc and kata container runtime. In *2020 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, Jul 2020.

[67] Hugo Lefeuvre, Gaulthier Gain, Vlad-Andrei Bădoiu, Daniel Dinca, Vlad-Radu Schiller, Costin Raiciu, Felipe Huici, and Pierre Olivier. Loupe: Driving the development of os compatibility layers. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '24, page 249–267, New York, NY, USA, 2024. Association for Computing Machinery.

[68] Dingji Li, Zeyu Mi, Yubin Xia, Binyu Zang, Haibo Chen, and Haibing Guan. Twinvisor: Hardware-isolated confidential virtual machines for arm. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 638–654, New York, NY, USA, 2021. Association for Computing Machinery.

[69] Feifei Li. Cloud-native database systems at alibaba: opportunities and challenges. *Proc. VLDB Endow.*, 12(12):2263–2272, August 2019.

[70] Mingyu Li, Yubin Xia, and Haibo Chen. Confidential serverless made efficient with plug-in enclaves. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 306–318, 2021.

[71] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. RunD: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 53–68, Carlsbad, CA, July 2022. USENIX Association.

[72] Costin Lupu, Andrei Albiundefinedoru, Radu Nichita, Doru-Florin Blânzeanu, Mihai Pogonaru, Răzvan Deaconescu, and Costin Raiciu. Nephele: Extending virtualization environments for cloning

unikernel-based vms. In *Proceedings of the Eighteenth European Conference on Computer Systems*, EuroSys '23, page 574–589, New York, NY, USA, 2023. Association for Computing Machinery.

[73] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. Unikernels: library operating systems for the cloud. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, page 461–472, New York, NY, USA, 2013. Association for Computing Machinery.

[74] Filipe Manco, Costin Lupu, Florian Schmidt, Jose Mendes, Simon Kuenzer, Sumit Sati, Kenichi Yasukata, Costin Raiciu, and Felipe Huici. My vm is lighter (and safer) than your container. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, page 218–233, New York, NY, USA, 2017. Association for Computing Machinery.

[75] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: A microkernel approach to host networking. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 399–413, New York, NY, USA, 2019. Association for Computing Machinery.

[76] Dirk Merkel et al. Docker: lightweight linux containers for consistent development and deployment. *Linux j*, 239(2):2, 2014.

[77] Microsoft. Sidecar pattern - azure architecture center. https://learn.microsoft.com/en-us/azure/architecture/patterns/sidecar, 2024. Accessed: 2025-01-08.

[78] Netflix. Netflix open source software center. https://netflix.github.io/, 2023. Accessed: 2025-05-15.

[79] nghttp2.org. h2load - HTTP/2 benchmarking tool. [Online; accessed 2025-01-12].

[80] Zhixiong Niu, Hong Xu, Peng Cheng, Qiang Su, Yongqiang Xiong, Tao Wang, Dongsu Han, and Keith Winstein. NetKernel: Making network stack part of the virtualized infrastructure. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 143–157. USENIX Association, July 2020.

[81] Pierre Olivier, Daniel Chiba, Stefan Lankes, Changwoo Min, and Binoy Ravindran. A binary-compatible unikernel. In *Proceedings of the 15th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE 2019, page 59–73, New York, NY, USA, 2019. Association for Computing Machinery.

[82] Oracle Corporation. Mysql: The world's most popular open source database, 2025. Accessed: 2025-09-13.

[83] Linux Kernel Organization. Credentials. https://docs.kernel.org/security/credentials.html, January 2024. Accessed: 2024-04-15.

[84] Yicheng Ouyang, Kailai Shao, Kunqiu Chen, Ruobing Shen, Chao Chen, Mingze Xu, Yuqun Zhang, and Lingming Zhang. Mirrortaint: Practical non-intrusive dynamic taint tracking for jvm-based microservice systems. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 2514–2526. IEEE Press, 2023.

[85] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. Arrakis: The operating system is the control plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 1–16, Broomfield, CO, October 2014. USENIX Association.

[86] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. Rethinking the library os from the top down. In *Proceedings of the sixteenth international conference on Architectural support for programming languages and operating systems*, Mar 2011.

[87] Redis. Redis. https://redis.io/, 2021. Accessed June 13, 2024.

[88] Redis. Redis documentation - optimization: Benchmarks. https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/, 2024. Accessed: 2024-04-05.

[89] Rusty Russell. virtio: towards a de-facto standard for virtual i/o devices. *SIGOPS Oper. Syst. Rev.*, 42(5):95–103, jul 2008.

[90] Youren Shen, Hongliang Tian, Yu Chen, Kang Chen, Runji Wang, Yi Xu, Yubin Xia, and Shoumeng Yan. Occlum: Secure and efficient multitasking inside a single enclave of intel sgx. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 955–970, New York, NY, USA, 2020. Association for Computing Machinery.

[91] Zhiming Shen, Zhen Sun, Gur-Eyal Sela, Eugene Bagdasaryan, Christina Delimitrou, Robbert Van Renesse, and Hakim Weatherspoon. X-containers: Breaking down barriers to improve performance and isolation of cloud-native containers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '19, page 121–135, New York, NY, USA, 2019. Association for Computing Machinery.

[92] SOFAStack. Sofaboot: A framework for building spring boot applications with sofastack. https://github.com/sofastack/sofa-boot, 2025. Accessed: 2025-05-15.

[93] Solo.io. What is istio ambient mode? https://www.solo.io/topics/istio/ambient-mode, 2024. Accessed: 2025-05-04.

[94] Enge Song, Yang Song, Chengyun Lu, Tian Pan, Shaokai Zhang, Jianyuan Lu, Jiangu Zhao, Xining Wang, Xiaomin Wu, Minglan Gao, Zongquan Li, Ziyang Fang, Biao Lyu, Pengyu Zhang, Rong Wen, Li Yi, Zhigang Zong, and Shunmin Zhu. Canal mesh: A cloud-scale sidecar-free multi-tenant service mesh architecture. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 860–875, New York, NY, USA, 2024. Association for Computing Machinery.

[95] Spotify. Apollo: Spotify's graphql framework. https://github.com/spotify/apollo, 2023. Accessed: 2025-05-15.

[96] The New Stack. How airbnb and twitter cut back on microservice complexities. https://thenewstack.io/how-airbnb-and-twitter-cut-back-on-microservice-complexities/, 2021. Accessed: 2025-05-15.

[97] Kun Tian, Yu Zhang, Luwei Kang, Yan Zhao, and Yaozu Dong. coIOMMU: A virtual IOMMU with cooperative DMA buffer tracking for efficient memory management in direct I/O. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 479–492. USENIX Association, July 2020.

[98] Olufogorehan Tunde-Onadele, Jingzhu He, Ting Dai, and Xiaohui Gu. A study on container vulnerability exploit detection. In *2019 IEEE International Conference on Cloud Engineering (IC2E)*, pages 121–127, 2019.

[99] R. Uhlig, G. Neiger, D. Rodgers, A.L. Santoni, F.C.M. Martins, A.V. Anderson, S.M. Bennett, A. Kagi, F.H. Leung, and L. Smith. Intel virtualization technology. *Computer*, 38(5):48–56, 2005.

[100] Virtio-fs. Virtio-fs - a shared file system for virtual machines. https://virtio-fs.gitlab.io/, 2024. Accessed: 2024-06-20.

[101] CarlA Waldspurger. Memory resource management in vmware esx server.

[102] Gil Walker. wrk: A modern http benchmarking tool. https://github.com/wg/wrk, 2024. Accessed: 2024-04-05.

[103] Xingyu Wang, Junzhao Du, and Hui Liu. Performance and isolation analysis of runc, gvisor and kata containers runtimes. *Cluster Computing*, page 1497–1513, Apr 2022.

[104] Yaohui Wang, Ben Luo, and Yibin Shen. Efficient memory overcommitment for I/O passthrough enabled VMs via fine-grained page meta-data management. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 769–783, Boston, MA, July 2023. USENIX Association.

[105] Dan Williams, Ricardo Koller, Martin Lucina, and Nikhil Prakash. Unikernels as processes. In *Proceedings of the ACM Symposium on Cloud Computing*, SoCC '18, page 199–211, New York, NY, USA, 2018. Association for Computing Machinery.

[106] Gao Xiang. Enhanced read-only file system (erofs). https://www.kernel.org/doc/html/latest/filesystems/erofs.html. Accessed: 2024-06-17.

[107] Jietao Xiao, Nanzi Yang, Wenbo Shen, Jinku Li, Xin Guo, Zhiqiang Dong, Fei Xie, and Jianfeng Ma. Attacks are forwarded: Breaking the isolation of MicroVM-based containers through operation forwarding. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 7517–7534, Anaheim, CA, August 2023. USENIX Association.

[108] Nanzi Yang, Wenbo Shen, Jinku Li, Yutian Yang, Kangjie Lu, Jietao Xiao, Tianyu Zhou, Chenggang Qin, Wang Yu, Jianfeng Ma, and Kui Ren. Demons in the shared kernel: Abstract resource attacks against os-level virtualization. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021.

[109] EthanG. Young, Pengfei Zhu, Tyler Caraza-Harter, AndreaC. Arpaci-Dusseau, and RemziH. Arpaci-Dusseau. The true cost of containing: A gvisor case study. *USENIX conference on Hot Topics in Cloud Ccomputing,USENIX conference on Hot Topics in Cloud Ccomputing*, Jan 2019.

[110] Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. The demikernel datapath os architecture for microsecond-scale datacenter systems. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 195–211, New York, NY, USA, 2021. Association for Computing Machinery.

[111] Lingjun Zhu, Yifan Shen, Erci Xu, Bo Shi, Ting Fu, Shu Ma, Shuguang Chen, Zhongyu Wang, Haonan Wu, Xingyu Liao, Zhendan Yang, Zhongqing Chen, Wei Lin, Yijun Hou, Rong Liu, Chao Shi, Jiaji Zhu, and Jiesheng Wu. Deploying user-space TCP at cloud scale with LUNA. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 673–687, Boston, MA, July 2023. USENIX Association.

[112] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, XiongChun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, Danyang Zhuo, and Ratul Mahajan. Dissecting overheads of service mesh sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, SoCC '23, page 142–157, New York, NY, USA, 2023. Association for Computing Machinery.

Appendix A Gofer in gVisor

Gofer [40] in gVisor is responsible for providing filesystem operation proxies for the Sentry kernel [41], as shown in Figure 4. It is a standard host user process initiated with each container, communicating with the Sentry via the 9P protocol [1] over a socket or shared memory channel. The Sentry process runs in a restricted `seccomp` container without direct access to filesystem resources in host kernel, while the Gofer mediates all access to these resources, improving isolation to a certain extent.

The main challenge with the Gofer architecture is its performance impact: each message exchange between the Sentry kernel and Gofer involves at least one syscall from Sentry and therefore at least one `VM-exit`, with a 9P command requiring at least two messages (a request and a response).

To reduce this overhead, Gofer distinguishes between `sensitive file operations` and `low-risk file I/Os`. For sensitive operations such as opening, modifying, and deleting files in the host kernel by applications within containers, they still require auditing and authorization through Gofer's file I/O proxy. While for files in host kernel which have been granted accessibility by Gofer proxy, the Gofer can send host file descriptors to the Sentry via Unix domain sockets' `SCM_RIGHTS` feature. The Sentry can then directly operate on these file descriptors through syscalls to the host kernel, enabling the Sentry to avoid unnecessary IPC and 9P overhead for granted file access while still maintaining access control in the Gofer. This pathway can be referred as `low-risk file I/O`. This also has the benefit that the Sentry can map such file descriptors directly into application and Sentry's address spaces, leveraging the host kernel page cache and obviating the need to perform page caching for such files in the Sentry.

Appendix B Application's Syscalls

We profile the number of syscalls in one of our production applications. The testbed uses the same application described in subsection 2.1. As a reference, we separately counted the syscall requests of runc and gVisor, with an offered load of 220 TPS. Under this load, runc's CPU utilization was 42.95%, and gVisor's CPU utilization was 57.19%. The RT latencies (P90) were 52.95ms and 142.27ms, respectively.

Table 11. Syscall category.

Category	Syscalls
File	*sys_write*, *sys_read*, *sys_writev*, *sys_close*, *sys_open*, *sys_newstat*, *sys_newfstat*, *sys_access*
Network	*sys_recvfrom*, *sys_sendto*, *sys_setsockopt*, *sys_epoll_wait*, *sys_epoll_pwait*, *sys_poll*, *sys_select*
Memory	*sys_mmap*, *sys_munmap*, *sys_mprotect*, *sys_brk*, *sys_mincore*
Sync	*sys_futex*
Misc	*sys_nanosleep*, *sys_rt_sigaction*, *sys_rt_sigprocmask*, *sys_ioctl*, *sys_geteuid*, *sys_getegid*, *sys_getgid*, *sys_getuid*

We measure the number of system calls issued per second by each application. The classification results are summarized in Table 4, the classification criteria are defined in Table 11, and the detailed results are provided in Table 12.

Table 12. Application's syscalls (/s).

Syscall	runc	gVisor	Syscall	runc	gVisor
sys_futex	77,459	54,453	*sys_newstat*	228	209
sys_write	23,137	23,866	*sys_newfstat*	-	179
sys_epoll_wait	32,708	23,550	*sys_rt_sigaction*	-	141
sys_ioctl	18,384	18,522	*sys_rt_sigprocmask*	-	90
sys_read	18,271	17,622	*sys_mincore*	-	86
sys_poll	9,488	9,244	*sys_select*	-	67
sys_recvfrom	9,413	9,244	*sys_access*	-	59
sys_writev	5,815	5,754	*sys_munmap*	-	46
sys_sendto	4,727	4,624	*sys_geteuid*	-	36
sys_epoll_pwait	-	3,356	*sys_getegid*	-	34
sys_mmap	449	390	*sys_getgid*	-	34
sys_close	397	344	*sys_getuid*	-	34
sys_open	-	304	*sys_brk*	-	30
sys_mprotect	-	249	*sys_setsockopt*	-	22
sys_nanosleep	-	240			
Total	200,476	172,829			

Appendix C gVisor's VMCALLs

gVisor invokes host kernel syscalls via VMCALLs. We counted the types and numbers of those VMCALLs. The application and load levels are identical to those in Appendix B. The classification results are summarized in Table 5b, the criteria for classification are also defined in Table 11, and the detailed results are provided in Table 13.

Table 13. Statistics of various VMCALLs.

Syscall	Counts (/s)	Syscall	Counts (/s)
sys_futex	36,634	*sys_recvmsg*	4
sys_write	5,927	*sys_clock_gettime*	3
sys_nanosleep	4,888	*sys_fcntl*	1
sys_fallocate	688	*sys_sendmsg*	1
sys_pread64	237	*sys_accept*	1
sys_sched_yield	166	*sys_close*	1
sys_tgkill	54	*sys_epoll_pwait*	1
sys_newfstat	40	*sys_ppoll*	1
sys_mmap	29	*sys_pwrite64*	1
sys_munmap	29		
Total	48,729		